

14. CODEIGNITER

1. Introducción

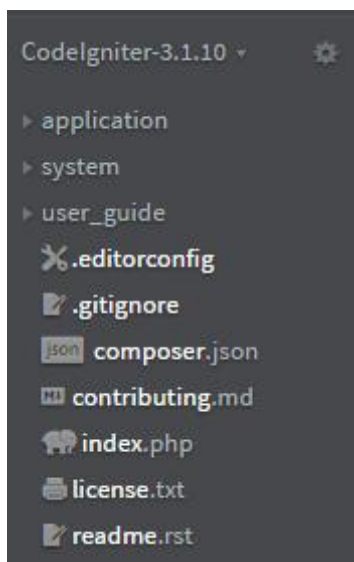
CodeIgniter es un framework de código abierto para crear aplicaciones web utilizando arquitectura MVC lo que facilita la organización del código y la separación entre el acceso a datos, la lógica de negocio y la capa de presentación de nuestra aplicación.

2. Instalación de CodeIgniter

Hay que tener instalado tanto PHP como Apache

- Descarga la última versión de CodeIgniter de la página oficial (versión 3)
- Descomprime el zip que has descargado, copia su contenido en la carpeta htdocs del xampp

Ya has instalado CodeIgniter y está listo para empezar a trabajar con él. La estructura de tu proyecto debe haber quedado así:



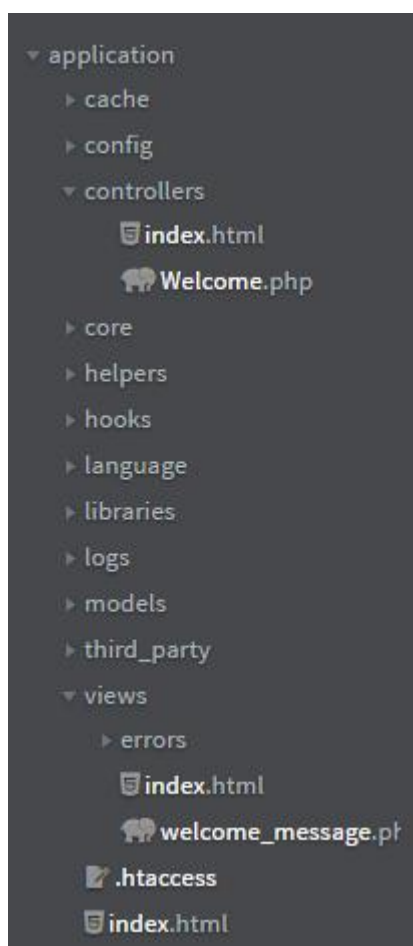
application/ : carpeta con los ficheros de nuestra aplicación, normalmente trabajaremos aquí. Aquí tendremos modelos, controladores y vistas (nuestros)

system/ : contiene las clases de CodeIgniter, lo que hace que funcione.

user_guide/ : aquí dentro tenemos documentación y un tutorial

index.php : este fichero es el punto de inicio de nuestra web, sólo se utiliza para que el entorno cargue los recursos necesarios y haga las comprobaciones de rutas oportunas.

En la carpeta application tenemos 3 subcarpetas básicas: **modelo**, **vista** y **controlador**. Además:



Helpers: Se encuentran aquellas funciones que nos facilitarán la escritura de código. Codeigniter proporciona algunos, como facilitarnos la creación de formularios. Nosotros crearemos otros más adelante para comprenderlos mejor.

Hooks: En esta carpeta alteraríamos el comportamiento que tiene normalmente el framework sin tocar en los archivos los contenidos de la carpeta system. Nosotros lo usaremos para cambiar la seguridad y controlar la entrada de usuarios.

Libraries: La usaremos para la creación de funciones complicadas, que involucren varias tablas o que sencillamente no creamos conveniente alojarla en el modelo. La intención de esto es aislar la lógica de la aplicación con el acceso a la base de datos.

Third_party: Serían las librerías que nosotros nos descargamos para ampliar funcionalidades de PHP. Como podría ser leer PDF o generarlos, envío de correos... Se alojarían todos aquellos plugins creados por personas ajenas a nuestra aplicación.

Config: Se encuentran los archivos más comunes de configuración, en este tutorial veremos algunos.

3. Primeros pasos de configuración

Podemos utilizar los ficheros de configuración para facilitar algunas tareas:

En el archivo `application/config/routes.php` podemos determinar el controlador por defecto. Al escribir la URL del proyecto en el navegador, se ejecute ese archivo

```
$route['default_controller'] = 'welcome'
```

A la hora de llamar a nuestra aplicación desde el navegador, CodeIgniter lo hace de la siguiente manera:

`http://localhost/miproyecto/index.php/welcome/index`
└── URL Base ─┘ └── Controlador ┘ └── Función ┘

Además cambiaremos `$config["base_url"]`

```
$config["base_url"] = "http://localhost/ejem1MVC"
```

donde `ejem1MVC` será el nombre de nuestra carpeta, para que CI sepa cuál es el sitio que queremos ejecutar y nos muestre las páginas que generemos.

En el archivo `index.php` tenemos la siguiente línea

```
define('ENVIRONMENT', 'development');
```

Esto significa que estamos en modo desarrollo, la aplicación mostrará mensajes de errores que se vayan produciendo.

Cuándo la tengamos lista para subirla, deberíamos cambiar ese valor a `'production'`

También se deben actualizar las rutas del archivo `config.php` mencionadas arriba

Ejercicio 1 - Práctica_10

Un controlador es una clase, podemos por tanto escribir su constructor.

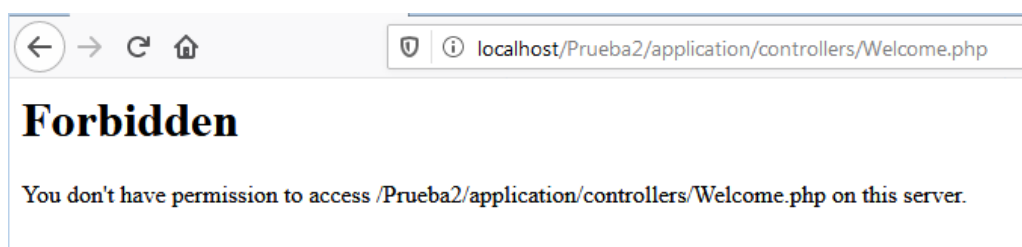
4. Flujo de aplicación de CodeIgniter

En CodeIgniter existe un procedimiento para atender una solicitud de página del cliente. Este proceso se realiza internamente, de manera transparente para nosotros.

La siguiente imagen (documentación del framework) muestra el proceso que sigue una solicitud



1. La solicitud de una página a partir de CodeIgniter comienza en un `index.php` que hay en la raíz del framework. En este index se cargan los controladores base y se definen algunas variables, por ejemplo 'BASEPATH'. Comprobando si esta variable está definida podemos asegurarnos de que la petición nos llega desde donde debe hacerlo y no accediendo directamente a nuestro controlador



2. Se realiza un filtrado de la URL para saber cuál es elemento que tiene que procesar esta página.

3. Si la página se había generado antes y está en la caché de CodeIgniter, se devuelve el archivo de la caché ya generado, con lo que se ahorra procesamientos repetidos. La caché se puede configurar y si lo deseamos, incluso deshabilitar.
4. El controlador adecuado realiza el procesamiento de la solicitud. CodeIgniter decide el controlador que debe procesar la solicitud en función de la URL solicitada.
5. El controlador comunica con una serie de módulos, los que necesite, para producir la página. A través de las vistas adecuadas, el controlador genera la página, tal cual se tiene que enviar al navegador.
6. Si la página no estaba en la caché, se introduce, para que las futuras solicitudes de esta página sean más rápidas.

Algunos de estos módulos, la caché o el enrutamiento, funcionan de manera transparente para nosotros. Otros (controladores, modelos y vistas) los tenemos que programar. Al estar separados nos ayudan a organizar nuestro código.

Tenemos a nuestra disposición librerías, ayudantes (helpers) y plugins ya escritos en CodeIgniter con clases y funciones muy útiles para el desarrollo de aplicaciones web.

5. URL

Una ventaja que tiene este framework es que genera URL amigables de forma transparente a nosotros, según vamos programando el sitio.

Cada una de las partes sirven para identificar qué controlador se va a hacer cargo del procesamiento de la página, así como de la función que se invocará y los parámetros que se le enviarán a la misma.

Por ejemplo:

`aplicacioncodeiginter.com/facturacion/editarempresa/5610`

- **aplicacioncodeiginter.com** es el nombre del supuesto dominio donde tenemos CodeIgniter instalado.
- **facturacion** es el nombre del controlador que se encargará de procesar la solicitud.
- **editarempresa** es el nombre del método que habrá dentro del controlador y donde estará el código que ejecute y genere la página
- **5610**, es el parámetro que se le pasa al método, que servirá en este caso para saber cuál es la empresa que se desea editar. Si necesitamos enviar varios parámetros, podremos colocarlos a continuación, separados por barras.

En nuestro caso, que ya hemos cambiado el controlador por defecto a uno nuestro, para llamar a su controlador welcome

<http://localhost/EjemploCodeIgniter/index.php/welcome/index>

Esta función no tiene parámetros

CodeIgniter pone a nuestra disposición una clase para trabajar con URLs llamada URI Class y una librería llamada URL Helper que contienen funciones para trabajar fácilmente con URLs y datos enviados en las mismas. En estas librerías hay funciones tan interesantes como **site_url()** que sirve para que el propio CodeIgniter cree una URL

dentro del sitio a partir de un parámetro que le pasemos. Otro ejemplo es **base_url()**, que simplemente devuelve la URL raíz donde está nuestra aplicación CodeIgniter. Para poder utilizar estas funciones hay que cargar el helper URL

`$this->load->helper('url')`

6. Enviando datos desde el controlador a la vista

Se genera un array asociativo con los datos que se quieren pasar, a cada dato se le asocia una clave. Dicha clave será utilizada (como una variable) por la vista para acceder al dato.

Ejemplo:

Cómo pasar datos a la vista desde el controlador

En el controlador:

```
public function index(){
    $datos=array('nombre'=>"Rosa");
    // $datos['nombre']="Rosa";
    $this->load->view('miaParametro',$datos);
}
```

En la vista:

```
<html>
    <head>
    </head>
    <body>
        <h1>Paso parámetros</h1>
        <?php
            echo "Hola, soy". $nombre;
        ?>
    </body>
</html>
```

7. HELPER FORM

Para el trabajo con formularios, el framework dispone del helperform, con métodos que me permiten generar la etiqueta (html) de apertura y cierre de un formulario. Dispone de funciones para generar los controles HTML

Para poder utilizar dichas funciones hay que cargar ese helper

- `$this->load->helper->('form')` en un controlador: estas funciones están disponibles en dicho controlador y las vistas que se carguen desde él.
- `$autoload['helper'] = array('form');` disponible en todo el proyecto

Estas son algunas de las funciones que tenemos disponibles

```
form_open( );           //create an opening form tag
form_label( );          //create a label
form_input( );          //create input field such as text , email etc.
form_password( );       //create a password input field
form_hidden( );         //create hidden field
form_radio( );          //create radio button
form_dropdown( );       //create select field
form_checkbox( );       //create checkbox
form_textarea( );       //create textarea
form_fieldset( );       //create fieldset
form_upload( );         //to upload files
form_submit( );         //create submit button
form_reset( );          //create reset button
form_close( );          //create form closing tag
set_value( );           //set default value for input tag
set_select( );          //set default value for select field
set_checkbox();         //set default value for checkbox
set_radio();            //set default value for radio button
```

Para generar el formulario: `form_open(action, array con el resto de datos)`

`form_open('Controlador/funcion') equivale <form action="ruta">`

*`$datos=array('class'=> 'miclase',
 'id'=>'identificador')`*

`form_open('Controlador/funcion', $datos)`

equivale a `<form action="ruta" class="miclase" id="identificador">`

Para cerrar form_close();

Más ejemplos

<code>echo form_input("caja");</code>	<code><input type="text" name=caja></code>
<code>\$data=array('name'=>'nombre', 'id'=>'id' 'placeholder'=>'Por favor, introduce nombre'); echo form_input(\$data);</code>	<code><input type="text" name="nombre" id="id" placeholder="Por favor, introduce nombre" /></code>
<code>\$data=array('name'=>'nombre', 'id'=>'id' 'value'=>'Rosa', 'readonly'=>'readonly'); echo form_input(\$data);</code>	<code><input type="text" name="nombre" value="Rosa" id="id" readonly="readonly" /></code>
<code>\$data_email = array('type' => 'email', 'name' => 'emp_email', 'id' => 'emp_email_id', 'placeholder' => 'Please Enter Email'); echo form_input(\$data_email);</code>	<code><input type="email" name="emp_email" value="" id="emp_email_id" placeholder="Please Enter Email" /></code>
<code>\$data_password = array('name' => 'password', 'id' => 'password_id', 'placeholder' => 'Please Enter Password'); echo form_password(\$data_password);</code>	<code><input type="password" name="password" value="" id="password_id" placeholder="Please Enter Password" /></code>
<code>form_hidden('employee_name', 'John');</code>	<code><input type="hidden" name="employee_name" value="John" /></code>

<pre> \$data_radio1 = array('name' => 'emp_marital_status', 'value' => 'Unmarried', 'checked' => TRUE,); \$data_radio2 = array('name' => 'emp_marital_status', 'value' => 'Married',); echo form_radio(\$data_radio1); echo form_radio(\$data_radio2); </pre>	<pre> <input type="radio" name="emp_marital_status" value="Unmarried" checked="checked" /> <input type="radio" name="emp_marital_status" value="Married" /> </pre>
<pre> \$data_gender = array('Male' => 'Male', 'Female' => 'Female'); echo form_dropdown('select', \$data_gender, 'Male'); </pre>	<pre> <select name="select"> <option value="Male" selected="selected">Male</option> <option value="Female">Female</option> </select> </pre>
<pre> \$data_checkbox1 = array('name' => 'qualification[]', 'value' => 'Graduation'); \$data_checkbox2 = array('name' => 'qualification[]', 'value' => 'Post Graduation'); echo form_checkbox(\$data_checkbox1); echo form_checkbox(\$data_checkbox2); </pre>	<pre> <input type="checkbox" name="qualification[]" value="Graduation" /> <input type="checkbox" name="qualification[]" value="Post Graduation" /> </pre>
<pre> <?php echo form_submit('submit', 'Submit'); </pre>	<pre> <input type="submit" name="submit" </pre>

?>	value="Submit" />
----	-------------------

8. Modelos

Nuestros modelos heredan de la clase `CI_Model` del framework. Se cargan en el controlador desde el que se llamará a sus funciones.

9. Recibiendo datos desde las vistas

El controlador utilizará la clase `input` para recibir a través de los métodos de la misma, los datos que llegan desde la vista (HTML GET, POST, COOKIES).

La clase, la carga automáticamente el sistema. Tienen dos funciones:

1. Aplicar medidas de seguridad a los datos que recibe

Filtrado de seguridad: el método de filtrado de seguridad se llama automáticamente cuando se invoca un nuevo controlador. Hace lo siguiente:

- Si `$config['allow_get_array']` es FALSE (el valor predeterminado es TRUE), destruye la matriz global GET.
- Destruye todas las variables globales en caso de que `register_globals` esté activado.
- Proporciona filtrado XSS (Cross-site Scripting Hacks evitar ataques de scripts entre sitios). Si queremos que el filtro se ejecute automáticamente cada vez que encuentre datos POST o COOKIE, lo habilitamos en `config.php`

```
$config['global_xss_filtering'] = TRUE;
```

o bien cada vez que recibimos un valor

```
$this->input->post('nombre', TRUE);
```

- Estandariza los caracteres de nueva línea a `PHP_EOL` (\n en sistemas operativos basados en UNIX, \r\n en Windows). Esto es configurable.

2. Proporcionar métodos para capturar dichos y procesarlos en los controladores

CodeIgniter viene con métodos de ayuda que nos permiten obtener elementos POST, GET, COOKIE o SERVER. La principal ventaja frente a ir directamente a un elemento (`$_POST['something']`) es que los métodos verificarán si el elemento está establecido y devolverán NULL si no lo están. Esto nos permite usar datos sin tener que comprobar si existen o están definidos.

```
If (isset($_POST['campo']))
```

```
else
```

Con los métodos integrados de CodeIgniter

```
$var=$this->input->post('campo')
```

Si `$var` es null el campo no está definido o es null

Los principales métodos son:

```
$this->input->post()
```

```
$this->input->get()
```

```
$this->input->cookie()
```

```
$this->input->server()
```

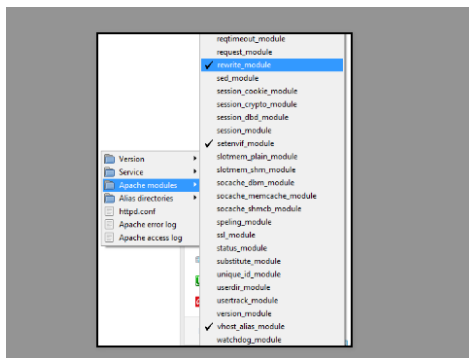
10. MÁS SOBRE CONFIGURACIÓN

Para prescindir del elemento *index.php* y que apache reescriba las urls, se incluye en la raíz del proyecto un fichero *.htaccess* con la siguiente configuración:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule^(.*)$/miSitio/index.php/$1 [L]
</IfModule>

<IfModule !mod_rewrite.c>
    ErrorDocument 404 /miSitio/index.php
</IfModule>
```

Además se activa el módulo rewrite de Apache (en wamp)



Y en el archivo *httpd.conf* (*wamp\bin\apache\apachex.x.conf*) en la línea 240 cambiamos *AllowOverrideNone* por *AllowOverrideall*

La redirección se hará a través de la misma URL, pero agregando "index.php/" después del nombre del dominio y antes de lo que haya justo después de ese nombre de dominio.

Ahora, sólo nos faltaría decirle a CodeIgniter que, al componer URLs para los enlaces entre las distintas páginas de nuestra aplicación web PHP, no coloque el *index.php*, pues no lo necesitamos. Esto lo podemos hacer desde el archivo de configuración principal, que están en: *system/application/config/config.php*

Simplemente tendríamos que editar la variable de configuración "index_page", poniéndola a la cadena vacía.

```
$config['index_page'] = '';
```

11. LIBRERÍA FORM_VALIDATION

Librería que contiene métodos para establecer reglas de validación en las entradas de los controles HTML

Hay que cargar la librería

```
$this->load->library('form_validation')
```

o bien cargarlas desde el autoload

```
$autoload['libraries'] = array('form_validation');
```

Para establecer una regla

```
$this->set-rules('nombre del control', 'regla de validación')
```

si hay varias reglas se separan con |

```
$this->form_validation->set_rules('edad', 'trim|required|integer');
```

Para ejecutar las reglas

```
$this->form_validation->run() retorna un booleano
```

Algunas reglas

- valid_email: Es la encargada de verificar si un campo input tiene el formato de correo electrónico (example@example.com).
- required: Es la encargada de verificar si un campo input no esté vacío.
- min_length, max_length: Se encarga de verificar la cantidad mínima y máxima de caracteres que puede tener un input (min_length[num], max_length[num]).
- integer: Se encarga de verificar que el valor de un campo input sea entero.

Se pueden personalizar los mensajes de error que proporciona el entorno

```
$this->form_validation->set_message('required', 'El campo %s es obligatorio')
```

el primer parámetro es la regla y el segundo la cadena a mostrar, dónde %s será el nombre del campo o campos que llevan asociada esa regla.

Se pueden escribir todos los mensajes de error generados en forma de lista

```
validation_errors()
```

O se pueden escribir uno a uno asociados a cada control HTML

```
form_error("nombre del control")
```

12. BBDD

Configuramos la base de datos en el archivo database.php, al menos los siguientes datos

```
$db['default']['hostname'] = "localhost";  
$db['default']['username'] = "DAW2";  
$db['default']['password'] = "2DAW2";  
$db['default']['database'] = "mibase";
```

Cargamos la librería database

```
$autoload['libraries'] = array('database'); Si lo hacemos en el autoload.php
```

```
$this->load->libraries('database') si lo hacemos en el controlador
```

Implementamos los modelos con los métodos para ejecutar los accesos a la base de datos.

CodeIgniter viene con una clase para base de datos, con todas las funciones y muy rápida que admite estructuras tradicionales (patrón active record). Las funciones de la base de datos ofrecen una sintaxis clara y simple.

Al cargar la librería database, estamos inicializando la clase **db** según la configuración establecida

Realizar consulta

```
$query = $this->db->query('SELECT name, title, email FROM my_table');
```

Métodos (\$query)

- result(): registros resultantes de la consulta en forma de filas
- num_rows(): número de registros obtenidos a partir de una select
- row(): fila actual

Si la sentencia SQL es INSERT, UPDATE, DELETE la clase db tiene un método affected_rows() que retorna el número de filas modificadas, añadidas o eliminadas

Ejemplos

```
foreach ($query->result() as $row)
{
    echo $row->title;
    echo $row->name;
    echo $row->email;
}
echo 'Total Results: ' . $query->num_rows();
```

```
$query = $this->db->query('SELECT name, title, email FROM my_table LIMIT 1');
$fila=$query->row()
echo $fila->title
```

```
$sql = "INSERT INTO table VALUES()";
$this->db->query($sql);
echo $this->db->affected_rows();
```

Escapar sentencias

La clase db nos proporciona tres métodos para escapar las sentencias SQL:

1. \$this->db->escape () Esta función solo escapa datos de cadena. También agrega automáticamente comillas simples alrededor de los datos para no tenerlas que poner nosotros

```
$sql= "INSERT INTO tabla (título) VALUES (" . $this->db->escape ( $ título ) . ")";
```


2. `$this->db->escape_str ()` Esta función escapa los datos independientemente del tipo.

```
$ sql= "INSERT INTO table (title) VALUES ('" . $this->db->escape_str( $title) ."')";
```

3. `$this->db->escape_like_str ()` Usaremos este método cuando la sentencia SQL debe contener este operador para que los comodines LIKE ('%', '_') se escapen correctamente.

```
$ search= '20% aumento ';  
$ sql= "SELECT id FROM tabla WHERE campo LIKE '%" .  
$this->db->escape_like_str( $search) . "% 'ESCAPE!' ";
```

Podemos ejecutar sentencias SQL abstrayendo dicho lenguaje, CodeIgniter dispone de la clase **QueryBuilder** que nos proporciona métodos para realizar operaciones sobre una bbdd sin escribir directamente las sentencias SQL. La gran ventaja de esta clase es que el sistema escapa las sentencias.

Seleccionar datos

Para consultas de selección podemos utilizar el método `get` en lugar del método `query`

`$this->db-> get ()`

```
$query = $this->db->get("entradas");  
// Ejecuta: SELECT * FROM entradas;
```

```
$query = $this->db->get("entradas", 10, 20);  
// Genera: SELECT * FROM entradas LIMIT 20,10
```

`$query` es un objeto similar al que maneja la clase `db`, por tanto el procesamiento del resultado es el mismo.

`$ this->db-> select (campos)`

```
$this->db->select ( 'título, autor, fecha' );  
$query = $this->db->select ( 'mytable' );  
// Ejecuta: SELECT título, autor, fecha FROM mytable
```

\$this->db->select_max (campo)

```
public function getNum($email){  
    $email=$this->db->escape($email);  
    $cadenaSQL="SELECT max(numIncidencia) from incidencia where email=".$email ;  
    $registros=$this->db->query($cadenaSQL);  
    $r=$registros->result_array();  
    $i=$r[0]['max(numIncidencia)'];  
    return $i;  
}
```

\$this->db->from()

```
$this->db->select ( 'título, autor, fecha' );  
$this->db->from( 'mytable');  
$query=$this->db->get();  
// Ejecuta: SELECT título, autor, fecha FROM mytable
```

\$this->db->where()

```
$query = $this->db->where("name","Rosa");  
$query = $this->db->where("password","a1234567");  
$query = $this->db->get("usuarios");  
// Ejecuta: SELECT * FROM entradas WHERE name = Rosa AND password = 'a1234567'
```

Podemos incluir condiciones en el nombre del campo

```
$ this->db->where ( 'nombre! =' , $nombre);  
$ this->db->where ( 'id <' , $id ); // WHERE nombre! = 'Rosa' AND id <45
```

Otros ejemplos:

```
$query = $this->db->distinct();  
$query = $this->db->get("entradas");  
// Genera: SELECT DISTINCT * FROM entradas
```

```
$query = $this->db->order_by("id","DESC");  
$query = $this->db->get("entradas");  
// Genera: SELECT * FROM entradas ORDER BYid DESC
```

Hay otras muchas funciones, consultar documentación.

Insertar, modificar, eliminar datos

Para otras consultas podemos utilizar los métodos insert, update, delete

```
$data = array(  
    'username' => 'Juan68',  
    'fname'    => 'Juan',  
    'lname'    => 'Pérez',  
    'register_date' => '2019-01-19'  
);  
$this->db->insert('users',$data);
```

También se le puede pasar un objeto, en ese caso inserta todos los campos definidos en la clase

```
class Myclass {  
    private $title;  
    private $content;  
    private $date;  
}  
obj=new Myclass('Mi título', 'Mi contenido', 'Mi fecha');  
  
$this->db->insert( 'mytable' , $ objeto );  
// Produce: INSERT INTO mytable (título, contenido, fecha) VALUES ('Mi título', 'Mi contenido',  
'Mi fecha')
```

```
$datos['id']=1;  
$this->db->delete('table',$datos)
```

```
$data = array(  
    'nombre' => 'aaaa',  
    'apellidos' => 'bbbbccccc',  
    'edad' => 'x'  
);  
$this->db->where('id', 3);  
$this->db->update(tabla, $data);
```

13. SESIONES

CodeIgniter dispone de una librería session, que nos permite guardar información que compartirán nuestras páginas. Esta librería nos permitirá trabajar con dos tipos de datos, permanentes y temporales. Los primeros son los que hemos usado hasta el momento, son guardados y mantenidos hasta que el usuario desea cerrar la sesión o la misma expira. Los datos temporales son aquellos que sirven para guardar un contenido que sólo es importante de una página a otra, pero luego deben ser borrados. Por ejemplo, al redireccionar de una página a otra, queremos guardar información en la primera para que pueda ser accedida desde la segunda.

Configurando la sesión

En primer lugar debemos cargar la librería con el nombre **session** bien en el constructor del controlador o en el fichero autoload

```
$autoload['libraries'] = array('database', 'session');  
$this->load->library('session')
```

Una vez cargada accedemos a ella con `$this->session`

También tendremos que ir a `application->config->config.php`, y buscar la línea:

```
$config['encryption_key'] = '';
```

Tendremos que poner una clave para nuestra sesión, por ejemplo:

```
$config['encryption_key'] = 'h0la9';
```

Cuando se carga una página, la clase de sesión verificará si el navegador del usuario envía una cookie de sesión válida. Si no coincide con una almacenada en el servidor o ha expirado, se creará y guardará una nueva sesión.

Si existe una sesión válida, su información se actualizará. Con cada actualización, la ID de sesión puede regenerarse si se configura para hacerlo.

Una vez inicializada, la clase Session se ejecuta automáticamente.

Para establecer las variables

```
$this->session->set_userdata("user", "Rosa") //escribir  
$this->session->userdata('user'); //leer
```

o bien prescindiendo de esas funciones

```
$this->session->user="Rosa"  
$n=$this->session->user
```

El método userdata retorna NULL si la variable no existe. También podemos usar el método has_userdata() para saber si una variable de sesión está definida

```
if ($this->session->has_userdata('user'))
```

Si necesitamos varios valores

```
$usuario_data = array(  
    'id' => $usuario->id,  
    'nombre' => $usuario->nombre,  
    'logueado' => true);  
$this->session->set_userdata($usuario_data);  
  
$this->session->userdata('logueado')
```

Si en lugar de utilizar arrays para almacenar varios datos queremos utilizar un objeto, necesitamos "serializarlo" para que luego sea reconocido como tal en la vista en que vayamos a usarlo. La función serialize() almacena como un string todos los datos de la clase, la función unserialize() recupera todos los datos de la clase a partir del string generado por la otra función.

Ejemplo:

```
$this->session->set_userdata('usuario', serialize($usu));  
$usu=unserialize($this->session->userdata('usuario'));
```

Para eliminar una variable de sesión

```
$this->unset_userdata('nombre')
```

`$this->session->sess_destroy()`: Destruye todas las variables de sesión. También puede usarse la función `session_destroy()`

Datos temporales

Datos de sesión que sólo están disponibles para la próxima solicitud. Para pasar un dato de una página a otra debemos usar el método **`set_flashdata()`** para setear valores, y **`flashdata()`** para recuperar ese valor temporal.

Al recargar la página esas variables se habrán eliminado

Sesiones en CodeIgniter 3

Las sesiones en CodeIgniter3 ofrecen nuevas funcionalidades. Han pasado de ser una librería a ser un driver para tener la opción entre otras cosas de escoger que tipo de sesiones queremos utilizar, nativas o cookies.

Métodos mágicos

- Gracias al método mágico **`__get`** podemos obtener una variable de sesión `userdata` por su nombre

```
$this->session->set_userdata("name", "Rosa")  
echo $this->session->name;
```

- El método **`has_userdata()`** nos devuelve un booleano, si existe la variable de sesión `true` en otro caso `false`.

```
if ( $this->session->has_userdata('user') )
```

- El método **`tempdata()`**

Nos permite setear una sesión durante un período de tiempo marcado en segundos, realmente útil.

```
public function set_tempdata()  
{  
    $this->session->set_tempdata('name', 'Uno de piera', 10);  
}
```

```
        redirect("home/get_tempdata");  
    }  
  
    public function get_tempdata()  
    {  
        echo $this->session->tempdata("name");  
    }  
}
```

Creamos una sesión que va a durar 10 segundos, pasados los 10 segundos la sesión es eliminada.

- Si necesitamos eliminar de forma explícita una sesión también podemos hacerlo con el método `unset_tempdata()`.

```
public function unset_tempdata()  
{  
    $this->session->unset_tempdata('name');  
}
```

14. Profiling your application

CodeIgniter dispone de la clase Profiler, que nos ayudará a depurar y optimizar nuestra aplicación. Proporciona información sobre resultados, consultas que se han ejecutado, datos POST, memoria utilizada, tiempo de ejecución empleado....

La clase tampoco necesita ser inicializada, se carga automáticamente si habilitamos la creación de perfiles en nuestro controlador.

```
$this->output->enable_profiler ( TRUE );
```

Al ejecutar la aplicación, se genera un informe que se mostrará en la parte inferior de nuestras páginas

POST DATA	
\$_POST[nombre]	sfadf
\$_POST[password]	dfadf
URI STRING	
loginController/isLogin	
CLASS/METHOD	
loginController/isLogin	
DATABASE: usuarios (LoginController.\$db) QUERIES: 1 (0.0086 seconds) (Hide)	
0.0086	SELECT * from usuarios where usuario='sfadf' and password='dfadf'
HTTP HEADERS (Show)	
CONFIG VARIABLES (Hide)	
base_url	http://localhost/Ejem6_db
index_page	index.php
uri_protocol	REQUEST_URI
url_suffix	
language	english
charset	UTF-8

Si queremos deshabilitarlo pasamos FALSE al método de arriba.

Podemos limitar las secciones de las que queremos información, queries, post, secciones,....

Key	Description	Default
benchmarks	Elapsed time of Benchmark points and total execution time	TRUE
config	CodeIgniter Config variables	TRUE
controller_info	The Controller class and method requested	TRUE
get	Any GET data passed in the request	TRUE
http_headers	The HTTP headers for the current request	TRUE
memory_usage	Amount of memory consumed by the current request, in bytes	TRUE
post	Any POST data passed in the request	TRUE
queries	Listing of all database queries executed, including execution time	TRUE
uri_string	The URI of the current request	TRUE
session_data	Data stored in the current session	TRUE
query_toggle_count	The number of queries after which the query block will default to hidden.	25

La clave es el parámetro que utilizaremos para configurar la sección. Utilizaremos el archivo profiler.php de la carpeta config

```
$config['queries'] = TRUE;
```

Habilita la depuración de todas las consultas a bbdd en toda la aplicación.

Si queremos habilitar varias secciones

```
$sections = array(
    'config' => TRUE,
    'queries' => TRUE
);

$this->output->set_profiler_sections($sections);
```

Podemos establecer puntos de referencia (BenchmarkPoints) para determinar el tiempo de ejecución o consumo de memoria de un conjunto de líneas, CodeIgniter dispone de una clase para esto Benchmarking que carga automáticamente.

La clase **Benchmark** se puede usar dentro de sus controladores, vistas o sus modelos. El proceso de uso es este:

1. Marcar un punto de inicio

2. Marcar un punto final
3. Ejecute la función "tiempo transcurrido" para ver los resultados

```
$ this->benchmark ->mark ( 'code_start' );
```

```
-----  
-----
```

```
$ this->benchmark ->mark ( 'code_end' );
```

```
$ this->benchmark ->elapsed_time( 'code_start' , 'code_end' );
```

Al ejecutar la aplicación

BENCHMARKS	
Loading Time: Base Classes	0.1033
Code	0.0002
Controller Execution Time (LoginController / IsLogin)	0.1131
Total Execution Time	0.2181

Ha añadido la línea Code que calcula el tiempo que tarda en ejecutar el código entre los puntos marcados.

14. ENCRYPTAR

Las funciones clásicas en php para encriptar son md5 o sha1. No son seguras y por eso se desarrollaron otras: hash

```
$nombre="Rosa";  
$encryp=sha1($nombre)  
$otro_encryp=md5($nombre)
```

Para que el método sea más seguro se utiliza una hash

hash(algoritmo, cadena a encryptar)

Para saber con qué algoritmos podemos encriptar a partir de la función **hash_algos()**

```
foreach(hash_algos() as $algoritmo)  
    echo $algoritmo. "-";  
  
$password="a1234567";  
foreach(hash_algos() as $algoritmo)  
    echo hash($algoritmo, $password);
```

Utilizar sólo un algoritmo para encriptar no es seguro porque a una cadena siempre le corresponde el mismo valor. Por esto aparecieron los cifrados que utilizan un algoritmo y una clave(cifrados simétricos o de clave privada).

En PHP existe una función **password_hash()** que crea un hash de similar seguridad al que crean algoritmos más seguros que los mencionados arriba.

```
password_hash(cadena, algoritmo, [coste])
```

donde coste es el número de veces que aplicaremos el algoritmo para encriptar repetidamente y obtener un hash más seguro

Ejemplo

```
password_hash($password, PASSWORD_DEFAULT)
```

Son encriptaciones en un solo sentido, no se descrypta la información. Si tengo una password encriptada y almacenada en una bbdd, para saber si coincide con la que

introduce el usuario utilizo la función **password_verify(\$password, \$hash)**. Retorna un booleano

CodeIgniter proporciona un par de método para encriptar utilizando el potente algoritmo AES

Para encriptar utilizamos la función **openssl_encrypt(cadena, algoritmo, clave)**

Para encriptar pasamos al método además de los datos la clave que utilizará el algoritmo para la encriptación. Para desencriptar pasamos el campo de la bbdd que queremos desencriptar y la clave que utilizamos para la encriptación.

Dicha clave la indicamos en el archivo config.php

```
$config['aes_encryption_key'] = '$U(U28J298_-|mkmjhuiyf!KMInxns?djlO¿?)9UWE8Y7urdv';
```

15. TEMPLATES

Librería que nos permite el uso de páginas “maestras”. Una página maestra es una página estándar dónde vendrán definidas secciones que utilizaremos en forma de variables

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<!--<meta name="description" content="<?php// echo $description;?>">
<meta name="keywords" content="<?php// echo $keywords;?>"-->
<title><?php echo $title;?></title>
<!--Cargo los archivos css-->
<?php echo $_styles ?>
</head>
<body>
<main>
<header id="cabecera" class="cabecera">
<?php echo $cabecera;?>
</header>
<section id="subcabecera" class="subcabecera">
<?php echo $subcabecera;?>
</section>
<section id="central" class="central">
<?php echo articles($noticias);?>
</section>
<footer id="footer" class="pie-pagina">
<span><strong>W3web | Studio</strong></span>
<span>Copyright 2016</span>
</footer>
</main>
<?php echo $_scripts ?>
<!--Cargo los archivos js-->
</body>
</html>
```